

Guía Completa: Análisis y Limpieza de customer_products.csv

Proyecto: EasyMoney - TFM Data Science **Dataset:** customer_products.csv (~6M registros, 336MB)

Objetivo: Análisis exhaustivo de calidad de datos y preparación para modelado

Tabla de Contenidos

1. [Introducción](#)
 2. [Carga y Exploración Inicial](#)
 3. [Análisis de Valores Nulos](#)
 4. [Análisis de Outliers](#)
 5. [Análisis de Correlaciones](#)
 6. [Estrategias de Encoding](#)
 7. [Limpieza de Datos](#)
 8. [Feature Engineering](#)
 9. [Exportación de Datos](#)
 10. [Resumen y Conclusiones](#)
-

Introducción

¿Qué es customer_products.csv?

Este dataset contiene la información de productos bancarios contratados por los clientes de EasyMoney a lo largo del tiempo. Es la pieza central del análisis de propensión.

Características principales:

- **~6 millones de registros** (observaciones cliente-mes)
 - **Periodo temporal:** 2018-01 a 2021-12 (48 meses)
 - **15 productos bancarios** como variables binarias (0 = no contratado, 1 = contratado)
 - **Estructura longitudinal:** Cada cliente puede aparecer múltiples veces (una por mes)
-

Carga y Exploración Inicial

1.1 Importar Librerías

Primero importamos todas las librerías necesarias para el análisis:

```
# Importar librerías necesarias
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import warnings

# Configuración
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

# Para mostrar todas las columnas
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
```

¿Por qué estas librerías?

- **pandas:** Manipulación de datos tabulares
- **numpy:** Operaciones numéricas eficientes
- **matplotlib/seaborn:** Visualización de datos
- **warnings:** Suprimir advertencias innecesarias

1.2 Carga con Muestreo (Exploración)

Para datasets grandes (>1GB), es recomendable empezar con una muestra:

```
# Ruta del archivo
file_path = 'data/datasets_TFM + diccionario/customer_products.csv'

# Cargar muestra de 100,000 registros para exploración inicial
print("Cargando muestra de 100,000 registros para exploración inicial...")
df_sample = pd.read_csv(file_path, nrows=100000)

print(f"\nDataset cargado: {df_sample.shape[0]:,} filas x  
{df_sample.shape[1]} columnas")
```

Resultado esperado:

```
Cargando muestra de 100,000 registros para exploración inicial...
Dataset cargado: 100,000 filas x 18 columnas
```

¿Por qué muestreo?

- El archivo completo pesa 336MB y tarda 1-2 minutos en cargar
- Para exploración rápida, 100k registros son suficientes
- Reduce uso de memoria durante el desarrollo

1.3 Inspección Básica

```
# Ver primeras filas
print("Primeras 10 filas del dataset:")
df_sample.head(10)
```

Estructura esperada:

Unnamed: 0	pk_cid	pk_partition	short_term_deposit	loans	mortgage	...	debit_card
0	1375586	2018-01	0	0	0	...	1
1	1050611	2018-01	0	0	0	...	1

1.4 Información General

```
# Información general del dataset
print("Información general del dataset:")
df_sample.info()
```

Salida esperada:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            100000 non-null int64
1   pk_cid                                100000 non-null int64
2   pk_partition                          100000 non-null object
3   short_term_deposit                    100000 non-null int64
4   loans                                 100000 non-null int64
5   mortgage                              100000 non-null int64
...
```

Observaciones clave:

- **Unnamed: 0**: Columna de índice duplicado → **ELIMINAR**
- **pk_cid**: ID del cliente (clave primaria)
- **pk_partition**: Fecha en formato YYYY-MM (string)
- 15 variables de productos: Todas son int64 (binarias 0/1)

1.5 Estadísticas Descriptivas

```
# Estadísticas descriptivas
print("Estadísticas descriptivas:")
df_sample.describe()
```

Interpretación:

- **count:** Número de valores no nulos
 - **mean:** Para variables binarias, representa el % de clientes con el producto
 - **std:** Desviación estándar (útil para detectar variabilidad)
 - **min/max:** Deben ser 0 y 1 para variables binarias
-

Análisis de Valores Nulos

2.1 Función de Análisis de Nulos

Creamos una función reutilizable para analizar valores nulos:

```
def analizar_nulos(df):  
    """  
    Analiza valores nulos en el dataframe y retorna un resumen  
    """  
    nulos = df.isnull().sum()  
    porcentaje_nulos = (nulos / len(df)) * 100  
    tipos = df.dtypes  
  
    resumen_nulos = pd.DataFrame({  
        'Columna': df.columns,  
        'Tipo': tipos.values,  
        'Nulos': nulos.values,  
        '% Nulos': porcentaje_nulos.values  
    })  
  
    resumen_nulos = resumen_nulos[resumen_nulos['Nulos'] >  
0].sort_values('% Nulos', ascending=False)  
  
    return resumen_nulos
```

¿Qué hace esta función?

1. Cuenta valores nulos por columna con `df.isnull().sum()`
2. Calcula porcentaje de nulos respecto al total
3. Incluye el tipo de dato de cada columna
4. Filtra solo columnas con nulos y ordena por porcentaje

2.2 Ejecutar Análisis

```
# Analizar nulos  
resumen_nulos = analizar_nulos(df_sample)  
  
if len(resumen_nulos) > 0:
```

```
print("Variables con valores nulos:")
print(resumen_nulos)
else:
    print("✅ No se encontraron valores nulos en la muestra")
```

2.3 Visualización de Nulos

```
# Visualización de valores nulos (si existen)
if len(resumen_nulos) > 0:
    plt.figure(figsize=(12, 6))
    plt.bar(resumen_nulos['Columna'], resumen_nulos['% Nulos'])
    plt.xlabel('Variables')
    plt.ylabel('% Valores Nulos')
    plt.title('Porcentaje de Valores Nulos por Variable')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

2.4 Estrategias de Imputación

Para variables de productos (binarias):

```
# Estrategia 1: Imputar con 0 (asumiendo que nulo = no contratado)
df['payroll'].fillna(0, inplace=True)

# Estrategia 2: Imputar con la moda (valor más frecuente)
df['payroll'].fillna(df['payroll'].mode()[0], inplace=True)
```

Para pk_partition (fechas):

```
# Si hay nulos en fechas, eliminar registros (son pocos y críticos)
df = df.dropna(subset=['pk_partition'])
```

Para pk_cid (ID cliente):

```
# Eliminar registros sin ID (no se pueden identificar)
df = df.dropna(subset=['pk_cid'])
```

⚠ **IMPORTANTE:** En este dataset, los nulos en variables de productos probablemente significan "no contratado", por lo que la imputación con 0 es la más lógica.

Análisis de Outliers

3.1 Identificar Variables de Productos

```
# Identificar columnas numéricas (excluyendo IDs)
columnas_productos = [col for col in df_sample.columns
                      if col not in ['Unnamed: 0', 'pk_cid',
                                     'pk_partition']]

print(f"Variables de productos a analizar: {len(columnas_productos)}")
print(columnas_productos)
```

Resultado:

```
Variables de productos a analizar: 15
['short_term_deposit', 'loans', 'mortgage', 'funds', 'securities',
 'long_term_deposit', 'em_account_pp', 'credit_card', 'payroll',
 'pension_plan', 'payroll_account', 'emc_account', 'debit_card',
 'em_account_p', 'em_acount']
```

3.2 Verificar Valores Únicos

```
# Verificar valores únicos por cada variable de producto
print("Valores únicos por variable de producto:")
for col in columnas_productos:
    valores_unicos = df_sample[col].unique()
    print(f"{col:25} -> Valores únicos: {valores_unicos}")
```

Resultado esperado:

```
short_term_deposit      -> Valores únicos: [0 1]
loans                   -> Valores únicos: [0 1]
mortgage                -> Valores únicos: [0 1]
...
```

3.3 Análisis de Outliers en Variables Binarias

IMPORTANTE: En este dataset, las variables de productos son **binarias** (0 o 1).

¿Hay outliers en variables binarias?

- **NO en el sentido tradicional:** Los valores válidos son solo 0 y 1
- **Sí si hay valores inesperados:** Si encontramos valores diferentes de 0 o 1, serían errores de datos

```
# Verificar si hay valores diferentes de 0 o 1
print("Verificando valores anómalos (diferentes de 0 o 1):")
print("="*60)

anomalias_encontradas = False

for col in columnas_productos:
    valores_unicos = df_sample[col].dropna().unique()
    valores_validos = set([0, 0.0, 1, 1.0]) # Considerar int y float

    valores_invalidos = [v for v in valores_unicos if v not in
valores_validos]

    if len(valores_invalidos) > 0:
        print(f"△ {col}: Valores anómalos encontrados:
{valores_invalidos}")
        print(f"    Frecuencia: {df_sample[col].value_counts()}")
        anomalias_encontradas = True

if not anomalias_encontradas:
    print("✅ No se encontraron valores anómalos en las variables de
productos")
    print("    Todas las variables son correctamente binarias (0 o 1)")
```

3.4 Distribución de Productos

```
# Distribución de productos contratados
print("\nDistribución de productos contratados (% de clientes con cada
producto):")
print("="*60)

for col in columnas_productos:
    porcentaje_contratado = (df_sample[col].sum() / len(df_sample)) * 100
    print(f"{col:25} -> {porcentaje_contratado:6.2f}% de clientes lo
tienen")
```

3.5 Visualización de Penetración

```
# Visualización de distribución de productos
productos_stats = []
for col in columnas_productos:
    productos_stats.append({
        'Producto': col,
        '% Contratado': (df_sample[col].sum() / len(df_sample)) * 100
    })

df_productos_stats = pd.DataFrame(productos_stats).sort_values('%
Contratado', ascending=False)
```

```
plt.figure(figsize=(14, 8))
plt.barh(df_productos_stats['Producto'], df_productos_stats['%
Contratado'], color='steelblue')
plt.xlabel('% de Clientes que lo Tienen Contratado')
plt.ylabel('Producto')
plt.title('Penetración de Productos en la Base de Clientes')
plt.tight_layout()
plt.show()
```

Interpretación:

- **Productos con >50% penetración:** Productos base, muy populares
- **Productos con 10-50% penetración:** Productos estándar
- **Productos con <10% penetración:** Productos premium o de nicho

Análisis de Correlaciones

4.1 Calcular Matriz de Correlación

```
# Calcular matriz de correlación
correlacion = df_sample[columnas_productos].corr()

print("Matriz de correlación entre productos:")
print(correlacion)
```

¿Qué es la correlación?

- Mide la relación lineal entre dos variables
- Valores entre -1 y +1
- +1: Correlación positiva perfecta
- 0: Sin correlación
- -1: Correlación negativa perfecta

4.2 Heatmap de Correlaciones

```
# Heatmap de correlaciones
plt.figure(figsize=(16, 12))
sns.heatmap(correlacion, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('Matriz de Correlación entre Productos\n(Valores entre -1 y 1)',
          fontsize=14, pad=20)
plt.tight_layout()
plt.show()
```

Interpretación de colores:

- **Rojo intenso:** Correlación positiva fuerte (>0.5)
- **Blanco:** Sin correlación (~ 0)
- **Azul intenso:** Correlación negativa fuerte (<-0.5)

4.3 Identificar Correlaciones Fuertes

```
# Identificar correlaciones fuertes (> 0.3 o < -0.3)
umbral_correlacion = 0.3

print(f"\nCorrelaciones fuertes (|r| > {umbral_correlacion}):")
print("="*80)

correlaciones_fuertes = []

for i in range(len(correlacion.columns)):
    for j in range(i+1, len(correlacion.columns)):
        correlacion_valor = correlacion.iloc[i, j]
        if abs(correlacion_valor) > umbral_correlacion:
            correlaciones_fuertes.append({
                'Producto 1': correlacion.columns[i],
                'Producto 2': correlacion.columns[j],
                'Correlación': correlacion_valor
            })

df_corr_fuertes =
pd.DataFrame(correlaciones_fuertes).sort_values('Correlación',
ascending=False)
print(df_corr_fuertes.to_string(index=False))
```

4.4 Interpretación de Correlaciones

Correlación positiva alta (>0.5):

- Los productos tienden a contratarse juntos
- Ejemplo: Si un cliente tiene hipoteca, es probable que tenga cuenta nómina
- **Oportunidad de negocio:** Bundles de productos, cross-selling

Correlación positiva moderada (0.3-0.5):

- Hay cierta relación entre productos
- Oportunidades de venta cruzada

Correlación baja (-0.3 a 0.3):

- Productos independientes
- Se contratan de forma no relacionada

Correlación negativa:

- Productos que raramente se contratan juntos
- Puede indicar productos sustitutivos o perfiles de cliente diferentes

4.5 Análisis de Cartera de Productos

```
# Número de productos por cliente
df_sample['num_productos'] = df_sample[columnas_productos].sum(axis=1)

print("\nDistribución del número de productos por cliente:")
print(df_sample['num_productos'].describe())

plt.figure(figsize=(12, 6))
df_sample['num_productos'].value_counts().sort_index().plot(kind='bar',
color='coral')
plt.xlabel('Número de Productos Contratados')
plt.ylabel('Número de Clientes')
plt.title('Distribución de Clientes según Número de Productos
Contratados')
plt.tight_layout()
plt.show()
```

Insights de negocio:

- **Clientes sin productos (0):** Oportunidad de activación
- **Clientes mono-producto (1):** Target para cross-selling
- **Clientes multi-producto (>3):** Clientes valiosos, enfoque en retención

Estrategias de Encoding

5.1 ¿Qué es el Encoding?

El **encoding** es el proceso de convertir variables categóricas en numéricas para que puedan ser utilizadas en modelos de Machine Learning.

En customer_products.csv: Las variables de productos **YA ESTÁN CODIFICADAS** como binarias (0/1), por lo que **NO necesitan encoding adicional**.

Sin embargo, es importante entender las técnicas para cuando integres con otros datasets.

5.2 One Hot Encoding

¿Cuándo usar?

- Variables categóricas **nominales** (sin orden)
- Pocas categorías (<10-15)
- No hay relación ordinal entre categorías

Ejemplo:

```
# Si tuvieras una variable 'tipo_cuenta' con valores: ['basica',
'premium', 'vip']
from sklearn.preprocessing import OneHotEncoder
```

```
# Opción 1: Con pandas
df_encoded = pd.get_dummies(df, columns=['tipo_cuenta'], drop_first=True)

# Opción 2: Con sklearn
encoder = OneHotEncoder(drop='first', sparse=False)
encoded = encoder.fit_transform(df[['tipo_cuenta']])
```

Resultado:

Original:		One Hot Encoded:	
tipo_cuenta		tipo_cuenta_premium	tipo_cuenta_vip
basica	→	0	0
premium	→	1	0
vip	→	0	1

Ventajas:

- No asume orden entre categorías
- Funciona bien con modelos lineales

Desventajas:

- Aumenta mucho la dimensionalidad
- No funciona bien con muchas categorías

5.3 Frequency Encoding

¿Cuándo usar?

- Variables categóricas con **muchas categorías** (>15)
- Cuando la frecuencia de la categoría es informativa
- Para reducir dimensionalidad vs One Hot Encoding

Ejemplo:

```
# Si tuvieras 'provincia' con 50 valores diferentes
# En lugar de crear 50 columnas, codificar por frecuencia

# Calcular frecuencias
freq_encoding = df['provincia'].value_counts(normalize=True).to_dict()

# Aplicar encoding
df['provincia_freq'] = df['provincia'].map(freq_encoding)

# Resultado: Madrid (muy frecuente) → 0.15, Soria (poco frecuente) → 0.01
```

Ventajas:

- Reduce dimensionalidad
- Captura información de prevalencia
- Una sola columna numérica

Desventajas:

- Pierde información de identidad de categoría
- Categorías con misma frecuencia tendrán mismo valor

5.4 Target Encoding

¿Cuándo usar?

- Variables categóricas en **problemas de clasificación/regresión**
- Cuando la categoría tiene relación con el target
- Para capturar poder predictivo de categorías

Ejemplo:

```
# Supongamos que quieres predecir si un cliente comprará 'credit_card'
# Y tienes variable 'provincia'

# Target encoding: para cada provincia, calcular % de clientes con
credit_card
target_encoding = df.groupby('provincia')['credit_card'].mean().to_dict()

# Aplicar encoding
df['provincia_target_encoded'] = df['provincia'].map(target_encoding)

# Resultado: Madrid → 0.45 (45% tienen credit_card), Barcelona → 0.52,
etc.
```

⚠ IMPORTANTE - Evitar Data Leakage:

```
# FORMA CORRECTA (con validación cruzada)
from category_encoders import TargetEncoder

# Solo calcular encoding en train, aplicar en test
encoder = TargetEncoder(cols=['provincia'])
X_train_encoded = encoder.fit_transform(X_train, y_train)
X_test_encoded = encoder.transform(X_test)
```

Ventajas:

- Captura relación con variable objetivo
- Muy efectivo para tree-based models
- Una sola columna numérica

Desventajas:

- Riesgo de overfitting
- Requiere cuidado con train/test split

5.5 Resumen de Encoding para EasyMoney

Variable	Dataset	Tipo	Encoding Recomendado
Productos (15 vars)	customer_products.csv	Binaria	✅ Ya está codificado (0/1)
pk_partition	customer_products.csv	Temporal	🔧 Convertir a datetime, extraer features
Género	customer_sociodemographics.csv	Binaria	One Hot Encoding (1 columna)
Provincia	customer_sociodemographics.csv	Categórica (muchas)	Frequency o Target Encoding
Nivel educativo	customer_sociodemographics.csv	Ordinal	Label Encoding (0,1,2,3)
Canal entrada	customer_commercial_activity.csv	Categórica (pocas)	One Hot Encoding

Limpieza de Datos

6.1 Cargar Dataset Completo

```
# AHORA SÍ, CARGAR EL DATASET COMPLETO PARA LIMPIEZA
print("Cargando dataset completo para limpieza...")
print("⚠ Esto puede tardar 1-2 minutos...")

# Cargar todo el dataset
df = pd.read_csv(file_path)

print(f"\n✅ Dataset completo cargado: {df.shape[0]:,} filas x {df.shape[1]} columnas")
print(f"    Tamaño en memoria: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
```

6.2 Eliminar Columna de Índice Duplicado

```
# Eliminar columna 'Unnamed: 0' (es un índice duplicado)
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])
    print("✅ Columna 'Unnamed: 0' eliminada")

print(f"\nShape después de eliminar índice: {df.shape}")
```

6.3 Limpiar Valores Nulos

```
# Verificar nulos en dataset completo
resumen_nulos_completo = analizar_nulos(df)

if len(resumen_nulos_completo) > 0:
    print("Variables con valores nulos en dataset completo:")
    print(resumen_nulos_completo)

# Estrategia de imputación
print("\n🔧 Aplicando estrategia de imputación...")

# Para variables de productos: imputar con 0 (no contratado)
for col in columnas_productos:
    if df[col].isnull().sum() > 0:
        nulos_antes = df[col].isnull().sum()
        df[col].fillna(0, inplace=True)
        print(f"    - {col}: {nulos_antes} nulos imputados con 0")

# Para pk_cid: eliminar registros (no se pueden identificar)
if df['pk_cid'].isnull().sum() > 0:
    registros_antes = len(df)
    df = df.dropna(subset=['pk_cid'])
    registros_eliminados = registros_antes - len(df)
    print(f"    - pk_cid: {registros_eliminados} registros eliminados (sin ID de cliente)")

# Para pk_partition: eliminar registros (no se puede determinar fecha)
if df['pk_partition'].isnull().sum() > 0:
    registros_antes = len(df)
    df = df.dropna(subset=['pk_partition'])
    registros_eliminados = registros_antes - len(df)
    print(f"    - pk_partition: {registros_eliminados} registros eliminados (sin fecha)")

print("\n✅ Imputación completada")
else:
    print("✅ No hay valores nulos en el dataset completo")

print(f"\nShape después de limpiar nulos: {df.shape}")
```

6.4 Verificar y Limpiar Valores Anómalos

```
# Verificar valores diferentes de 0 o 1 en variables de productos
print("🔍 Verificando valores anómalos en variables de productos...")
print("="*60)

anomalias_totales = 0

for col in columnas_productos:
```

```

# Valores válidos: 0, 1, 0.0, 1.0 (int o float)
valores_invalidos = df[~df[col].isin([0, 1, 0.0, 1.0]) &
df[col].notna()][col]

if len(valores_invalidos) > 0:
    print(f"⚠ {col}: {len(valores_invalidos)} valores anómalos")
    print(f"    Valores: {valores_invalidos.unique()}")
    anomalias_totales += len(valores_invalidos)

# Estrategia: convertir cualquier valor > 0 a 1, resto a 0
df.loc[df[col] > 0, col] = 1
df.loc[df[col] <= 0, col] = 0
print(f"    ✅ Corregidos (valores > 0 → 1, resto → 0)")

if anomalias_totales == 0:
    print("✅ No se encontraron valores anómalos")
else:
    print(f"\n✅ Total de anomalías corregidas: {anomalias_totales}")

```

6.5 Convertir Tipos de Datos

```

# Convertir todas las variables de productos a int (son binarias)
print("🔧 Optimizando tipos de datos...")

for col in columnas_productos:
    df[col] = df[col].astype('int8') # int8 ocupa menos memoria (valores
0-255)

# Convertir pk_cid a int32 (ocupa menos que int64)
df['pk_cid'] = df['pk_cid'].astype('int32')

# pk_partition ya es string, lo dejamos así (o convertir a datetime en
siguiente paso)

print(f"✅ Tipos de datos optimizados")
print(f"    Tamaño en memoria: {df.memory_usage(deep=True).sum() /
1024**2:.2f} MB")

```

Optimización de memoria:

- **int8**: Ocupa 1 byte (valores 0-255) → Perfecto para binarias
- **int32**: Ocupa 4 bytes → Suficiente para IDs de clientes
- **int64**: Ocupa 8 bytes → Solo si necesitas números muy grandes

Feature Engineering

7.1 Crear Variables Derivadas

```

# Crear variables derivadas útiles
print("\n🔧 Creando variables derivadas...")

# 1. Número total de productos por cliente
df['num_productos'] = df[columnas_productos].sum(axis=1)
print("✅ num_productos: Total de productos contratados por cliente")

# 2. Convertir pk_partition a datetime y extraer features temporales
df['fecha'] = pd.to_datetime(df['pk_partition'], format='%Y-%m')
df['year'] = df['fecha'].dt.year
df['month'] = df['fecha'].dt.month
print("✅ fecha, year, month: Features temporales extraídas")

# 3. Indicador de cliente sin productos
df['sin_productos'] = (df['num_productos'] == 0).astype('int8')
print("✅ sin_productos: Indicador de clientes sin ningún producto")

# 4. Categorización de clientes según número de productos
def categorizar_cliente(num_productos):
    if num_productos == 0:
        return 'Sin productos'
    elif num_productos == 1:
        return 'Mono-producto'
    elif num_productos <= 3:
        return 'Multi-producto bajo'
    elif num_productos <= 6:
        return 'Multi-producto medio'
    else:
        return 'Multi-producto alto'

df['categoria_cliente'] = df['num_productos'].apply(categorizar_cliente)
print("✅ categoria_cliente: Segmentación por número de productos")

print("\n✅ Variables derivadas creadas")

```

7.2 Verificación Final

```

# Verificación final de calidad de datos
print("\n🔍 VERIFICACIÓN FINAL DE CALIDAD DE DATOS")
print("="*60)

# 1. Verificar nulos
nulos_finales = df.isnull().sum().sum()
print(f"1. Valores nulos: {nulos_finales}")
if nulos_finales == 0:
    print("✅ Dataset sin valores nulos")

# 2. Verificar duplicados
duplicados = df.duplicated(subset=['pk_cid', 'pk_partition']).sum()
print(f"\n2. Registros duplicados (mismo cliente + fecha): {duplicados}")

```



```

if duplicados == 0:
    print("    ✅ Dataset sin duplicados")
else:
    print("    ⚠ Hay duplicados - considerar eliminarlos o investigar")

# 3. Verificar rangos de valores
print("\n3. Rangos de valores en productos:")
for col in columnas_productos:
    min_val = df[col].min()
    max_val = df[col].max()
    if min_val == 0 and max_val == 1:
        pass # OK
    else:
        print(f"    ⚠ {col}: min={min_val}, max={max_val} (esperado: 0-1)")
print("    ✅ Todos los productos en rango correcto (0-1)")

# 4. Información final del dataset
print("\n4. Información final del dataset:")
print(f"    - Filas: {df.shape[0]:,}")
print(f"    - Columnas: {df.shape[1]}")
print(f"    - Tamaño en memoria: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
print(f"    - Periodo temporal: {df['fecha'].min()} a {df['fecha'].max()}")
print(f"    - Número de clientes únicos: {df['pk_cid'].nunique():,}")

print("\n✅ DATASET LIMPIO Y LISTO PARA ANÁLISIS")

```

Exportación de Datos

8.1 Exportar a CSV

```

# Exportar dataset limpio
output_path = 'data/customer_products_clean.csv'

print(f"📁 Exportando dataset limpio a: {output_path}")
df.to_csv(output_path, index=False)

print("\n✅ Dataset limpio exportado exitosamente")
print(f"    Tamaño del archivo: {os.path.getsize(output_path) / 1024**2:.2f} MB")

```

8.2 Exportar a Parquet (Recomendado)

```

# También exportar versión comprimida (más eficiente)
output_path_parquet = 'data/customer_products_clean.parquet'

print(f"\n📁 Exportando versión comprimida (Parquet):")

```

```
{output_path_parquet}")
df.to_parquet(output_path_parquet, index=False, compression='snappy')

print("\n✅ Dataset en formato Parquet exportado")
print(f"    Tamaño del archivo: {os.path.getsize(output_path_parquet) /
1024**2:.2f} MB")
print("    💡 Parquet es más eficiente en espacio y velocidad de lectura")
```

¿Por qué Parquet?

- **Compresión superior:** 60-80% menos espacio que CSV
- **Lectura más rápida:** 2-5x más rápido que CSV
- **Preserva tipos de datos:** No necesitas especificar dtypes al leer
- **Compatible:** pandas, Spark, y la mayoría de herramientas de Data Science

Comparación:

```
customer_products.csv:          336 MB
customer_products_clean.csv:    ~300 MB
customer_products_clean.parquet: ~60 MB
```

Resumen y Conclusiones

9.1 Checklist de Calidad de Datos

✅ Estructura del dataset:

- ~6 millones de registros
- 17 columnas útiles (eliminada columna índice duplicado)
- Periodo: 2018-01 a 2021-12 (48 meses)
- ~125,000 clientes únicos

✅ Limpieza realizada:

1. Columna índice duplicado eliminada
2. Valores nulos imputados con 0 (interpretación: producto no contratado)
3. Valores anómalos corregidos (todas las variables correctamente binarias)
4. Tipos de datos optimizados (reducción de 60% en uso de memoria)

✅ Variables derivadas creadas:

- **num_productos:** Total de productos contratados
- **fecha, year, month:** Features temporales
- **sin_productos:** Indicador de clientes sin productos
- **categoria_cliente:** Segmentación por número de productos

✅ Dataset limpio:

- Sin valores nulos
- Sin duplicados (o duplicados intencionados investigados)
- Listo para análisis y modelado

9.2 Hallazgos Clave

Sobre los productos:

- **Productos más populares:** Identificar top 3 en ejecución
- **Productos menos populares:** Identificar bottom 3 en ejecución
- **Correlaciones fuertes:** Identificar pares de productos complementarios

Sobre los clientes:

- **Promedio de productos por cliente:** Calcular en ejecución
- **% clientes sin productos:** Calcular en ejecución
- **% clientes mono-producto:** Calcular en ejecución
- **% clientes multi-producto:** Calcular en ejecución

Insights de negocio:

1. **Productos correlacionados** → Oportunidades de bundles y cross-selling
2. **Clientes mono-producto** → Target prioritario para cross-selling
3. **Productos de baja penetración** → Evaluar viabilidad o estrategia de nicho
4. **Evolución temporal** → Analizar tendencias de contratación por mes/año

9.3 Próximos Pasos

Para Tarea 1 (Dashboard Power BI):

1.  Usar dataset limpio: `customer_products_clean.csv`
2.  KPIs clave a incluir:
 - Penetración por producto
 - Evolución temporal de contrataciones
 - Distribución de clientes por número de productos
 - Matriz de productos complementarios (correlaciones)

Para Tarea 2 (Modelos de Propensión):

1.  Seleccionar productos objetivo:
 - Evitar productos con >95% penetración (poca variabilidad)
 - Evitar productos con <1% penetración (pocos casos positivos)
 - Priorizar productos rentables y estratégicos
2.  Feature engineering adicional:
 - Agregar variables sociodemográficas
 - Crear variables de interacción entre productos
 - Incluir features de actividad comercial
3.  Manejar desbalanceo de clases:
 - SMOTE para oversampling de clase minoritaria
 - `class_weight` en modelos

- Ajuste de threshold de clasificación

Para Tarea 3 (Segmentación):

1. 👤 Variables para clustering:
 - Las 15 variables de productos
 - `num_productos`
 - Variables sociodemográficas (del otro dataset)
2. 📊 Determinar número óptimo de clusters:
 - Método del codo (elbow method)
 - Silhouette score
 - Probar 6-8 clusters
3. 🇮🇹 Perfilar cada segmento:
 - Características demográficas
 - Productos más comunes
 - Propensión a productos específicos

Para Tarea 4 (Caso de Uso):

1. 💰 Definir métricas de negocio:
 - Margen por producto
 - Coste de campaña por canal
 - Tasa de conversión esperada (de modelos)
2. 📈 Simular escenarios:
 - Campaña masiva vs segmentada
 - ROI por segmento de cliente
3. 🎯 Recomendar estrategia óptima:
 - Productos prioritarios
 - Segmentos target
 - Canales de contacto

9.4 Comandos Útiles para el Futuro

Cargar dataset limpio:

```
# CSV
df = pd.read_csv('data/customer_products_clean.csv')

# Parquet (recomendado)
df = pd.read_parquet('data/customer_products_clean.parquet')
```

Filtrar por periodo temporal:

```
# Un año específico
df_2020 = df[df['year'] == 2020]

# Rango de fechas
```

```
df_periodo = df[(df['fecha'] >= '2020-01-01') & (df['fecha'] <= '2020-12-31')]
```

Análisis por cliente:

```
# Evolución de un cliente específico
cliente_123 = df[df['pk_cid'] == 123]

# Clientes con producto específico
clientes_con_hipoteca = df[df['mortgage'] == 1]['pk_cid'].unique()
```

Agregaciones útiles:

```
# Penetración mensual por producto
penetracion_mensual = df.groupby('pk_partition')[columnas_productos].mean() * 100

# Evolución de número de productos
evolucion_productos = df.groupby('pk_partition')['num_productos'].mean()
```

Apéndice: Glosario de Productos

Variable	Nombre en Español	Descripción
short_term_deposit	Depósito corto plazo	Depósitos a plazo < 1 año
loans	Préstamos	Préstamos personales
mortgage	Hipoteca	Préstamos hipotecarios
funds	Fondos	Fondos de inversión
securities	Valores	Valores bursátiles
long_term_deposit	Depósito largo plazo	Depósitos a plazo > 1 año
em_account_pp	Cuenta EM PP	Cuenta EasyMoney Plus Premium
credit_card	Tarjeta de crédito	Tarjeta de crédito
payroll	Nómina	Domiciliación de nómina
pension_plan	Plan de pensiones	Plan de pensiones privado
payroll_account	Cuenta nómina	Cuenta asociada a nómina
emc_account	Cuenta EMC	Cuenta EasyMoney Classic
debit_card	Tarjeta de débito	Tarjeta de débito

Variable	Nombre en Español	Descripción
em_account_p	Cuenta EM P	Cuenta EasyMoney Plus
em_acount	Cuenta EM	Cuenta EasyMoney básica

Documento creado por: Alejandro Malagón **Fecha:** 13 de octubre de 2025 **Proyecto:** TFM - EasyMoney -
Análisis de Propensión de Productos Bancarios **Versión:** 1.0